

1. INTRODUCTION TO DBMS & TYPES OF DATABASES

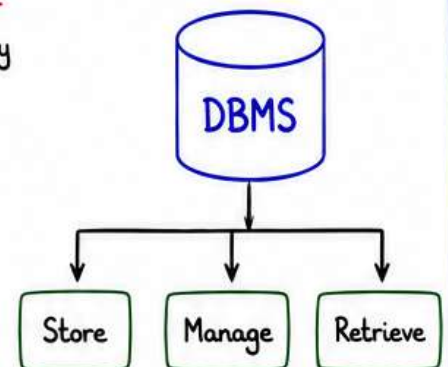
What is DBMS?

DBMS (Database Management System) is a software system that helps us to store, manage, retrieve and manipulate data in an organized and efficient way.

Examples: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB etc.

What does DBMS Provide?

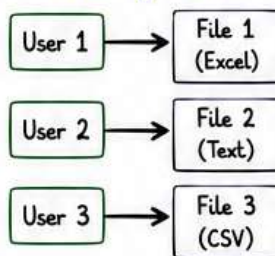
- Data organization and integrity
- Security and access control
- Concurrency control (multiple users at same time)
- Backup and recovery
- Data independence
- Efficient query processing
- Reduced data redundancy



Why DBMS is Used?

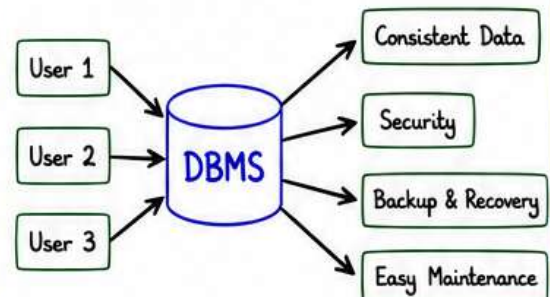
- Handling large amount of data
- Multiple users can access data simultaneously
- Data consistency and accuracy
- Security and privacy
- Easy maintenance and scalability
- Data independence

Without DBMS (File System)



✗ Redundant Data
Inconsistent, Hard to Manage

With DBMS



✓ Organized, Consistent
Easy to Manage

Types of Databases

1) SQL (Relational) Databases

- Data stored in tables (rows & columns)
- Schema is predefined
- Uses SQL for operations
- Strong consistency and ACID properties
- Examples: MySQL, PostgreSQL, Oracle, SQL Server

Student Table (Example)

ID	Name	Age	City
1	Ankit	22	Delhi
2	Neha	23	Mumbai
3	Ravi	21	Pune

2) NoSQL (Non-Relational) Databases

- Data stored in non-tabular formats
- Schema is flexible / dynamic
- Designed for large scale & distributed systems
- High availability and horizontal scalability
- Types: Key-Value, Document, Column-Family, Graph, Time-Series
- Examples: MongoDB, Cassandra, Redis, DynamoDB, Neo4j

Key-Value (Redis)

Key → Value
101 → Ankit
102 → Neha

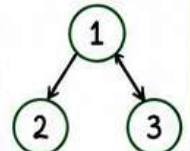
Document (MongoDB)

```
{
  "_id": 101,
  "name": "Ankit",
  "age": 22
}
```

Column-Family (Cassandra)

Row Key	Column1	Column2
1	A	B
2	C	D

Graph (Neo4j)



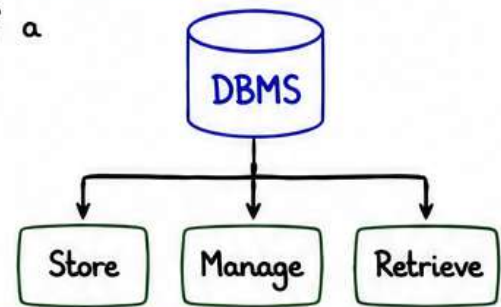
Key Takeaway: DBMS helps in efficient data management by providing structure, security, consistency and easy access.

2. DBMS ARCHITECTURE (1-TIER, 2-TIER, 3-TIER)

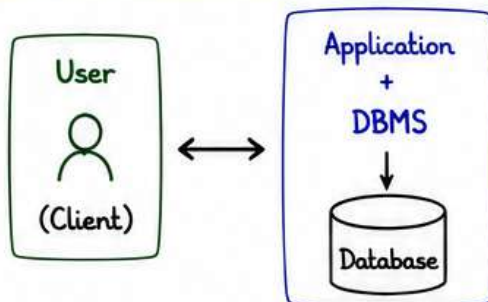
★ **DBMS Architecture** defines how components of a database system are organized and how they communicate with each other.

There are three main architectures:

1-Tier , 2-Tier , 3-Tier



1) 1-TIER ARCHITECTURE (SINGLE TIER)

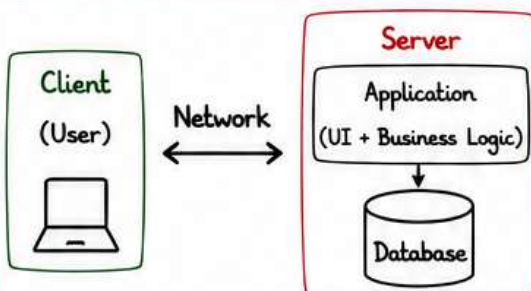


- Both the application logic and database reside on the same machine.
- Suitable for small systems and single user.
- Easy to develop and maintain.
- Example: MS Access

Pros & Cons

- + Simple
- + Fast (no network)
- Not suitable for multiple users
- Low security

2) 2-TIER ARCHITECTURE (CLIENT-SERVER)

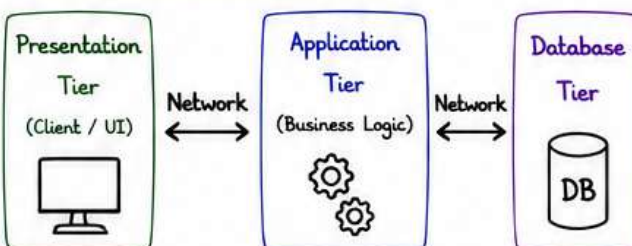


- Client sends request to the server.
- Server processes the request, executes business logic and accesses the database.
- Data is stored on the server.
- Example: Banking apps, Desktop Applications

Pros & Cons

- + Better security
- + Centralized data
- Heavy load on server
- Limited scalability

3) 3-TIER ARCHITECTURE



- Divides the system into three layers:
 - Presentation Tier → UI / Client
 - Application Tier → Business Logic
 - Database Tier → Data Storage
- Most widely used architecture.
- Example: Web applications (E-commerce, ERP, etc.)

Pros & Cons

- + Highly scalable
- + Better security
- + Easy maintenance
- More complex design
- Higher cost

COMPARISON SUMMARY

Feature	1-Tier	2-Tier	3-Tier
Layers	One	Two (Client + Server)	Three (Presentation + Application + DB)
Where DB Resides	Same machine	Server	Database Server
Network	Not required	Required	Required
Scalability	Low	Medium	High
Security	Low	Medium	High
Use Case	Small apps, Single user	Medium sized apps	Large scale, Web apps
Performance	High (small data)	Medium	High (optimized)



Key Takeaway : As the architecture moves from 1-Tier → 2-Tier → 3-Tier, we get better scalability, security and maintainability.

3. KEYS IN DBMS

(PRIMARY, FOREIGN, CANDIDATE, COMPOSITE, SUPER)

★ What is a Key?

A key is an attribute or a set of attributes that uniquely identifies a record (row) in a table. Keys help in maintaining data integrity and relationships between tables.

STUDENT Table (Example)

RollNo	Name	Email	Phone	DeptID
101	Ankit	ankit@gmail.com	9876543210	10
102	Neha	neha@gmail.com	9876543211	20
103	Ravi	ravi@gmail.com	9876543212	10

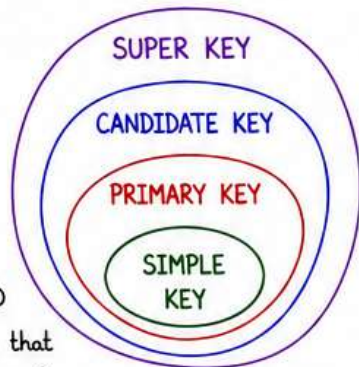
1 SUPER KEY

A set of one or more attributes that can uniquely identify a record in a table.

Example :

In STUDENT table:

- RollNo
- Email
- RollNo, Email
- RollNo, Phone
- RollNo, Email, DeptID
- (And any combination that uniquely identifies a row)



(Relationship between Keys)

2 CANDIDATE KEY

A minimal super key. None of its proper subsets can uniquely identify a record.

Example :

In STUDENT table:

- RollNo
- Email

(Both can uniquely identify a record and neither has extra unnecessary attributes)

3 PRIMARY KEY

A candidate key that is chosen by the developer to uniquely identify records.

Example :

RollNo (chosen as Primary Key)

- It must be UNIQUE and NOT NULL.
- Only one primary key in a table.

4 FOREIGN KEY

A key in one table that refers to the primary key of another table.

Example :

DEPARTMENT Table

DeptID	DeptName
10	IT
20	HR
30	Finance

STUDENT Table

RollNo	Name	DeptID (FK)
101	Ankit	10
102	Neha	20
103	Ravi	10

DeptID in STUDENT table is a FOREIGN KEY that refers to DeptID (Primary Key) in DEPARTMENT table.

5 COMPOSITE KEY

A key made up of two or more attributes together to uniquely identify a record.

Example :

In ENROLLMENT table:

RollNo (FK)	CourseID (FK)	EnrollDate
101	C101	01-01-2024
101	C102	15-01-2024

(RollNo, CourseID) together make a COMPOSITE KEY (since a student can enroll in multiple courses).



KEY TAKEAWAY

1. Super Key → Any set of attributes that uniquely identifies a record.
2. Candidate Key → Minimal Super Key.
3. Primary Key → Chosen Candidate Key (Unique & Not Null).
4. Foreign Key → Refers to Primary Key of another table.
5. Composite Key → Combination of two or more attributes.

4. NORMALIZATION IN DBMS

What is Normalization ?

Normalization is a technique used to organize data in a database to **reduce redundancy** and **improve data integrity**.

Goals of Normalization

- ✓ Minimize data redundancy
- ✓ Avoid update anomalies
- ✓ Ensure data consistency
- ✓ Improve data integrity



Clean Data
Consistent Data
Reliable Data

★ UNNORMALIZED TABLE (Not in 1NF)

StudentID	StudentName	Courses	Instructor	Phone
1	Ankit	DBMS, OS	Ravi, Mohan	9876543210
2	Neha	DBMS	Ravi	9876543211
3	Ravi	OS, CN	Mohan, Sita	9876543212

Problems :

- Multiple values in a single cell (Courses, Instructor)
- Redundancy of data
- Difficult to update and maintain
- Insertion, Update and Deletion anomalies

1NF (First Normal Form)

StudentID	StudentName	Course	Instructor	Phone
1	Ankit	DBMS	Ravi	9876543210
1	Ankit	OS	Mohan	9876543210
2	Neha	DBMS	Ravi	9876543211
3	Ravi	OS	Mohan	9876543212
3	Ravi	CN	Sita	9876543212

Rule :

Eliminate repeating groups.
Each column must have atomic (single) values.

Benefit :

- Removes repeating groups
- Each row is unique

2NF (Second Normal Form)

STUDENT		COURSE		ENROLLMENT	
StudentID	StudentName	CourseID	CourseName	StudentID	CourseID
1	Ankit	C1	DBMS	1	C1
2	Neha	C2	OS	1	C2
3	Ravi	C3	CN	2	C1
				3	C2
				3	C3

Rule :

Remove partial dependency.
Every non-key attribute must depend on the whole primary key.

Benefit :

- No partial dependency
- Better data organization

3NF (Third Normal Form)

STUDENT			COURSE		ENROLLMENT	
StudentID	StudentName	Phone	CourseID	CourseName	StudentID	CourseID
1	Ankit	9876543210	C1	DBMS	1	C1
2	Neha	9876543211	C2	OS	1	C2
3	Ravi	9876543212	C3	CN	2	C1
					3	C2
					3	C3

Rule :

Remove transitive dependency.
Non-key attributes should not depend on other non-key attributes.

Benefit :

- Removes transitive dependency
- More consistent and reliable data

NORMAL FORMS SUMMARY

Normal Form	Focus	Requirement	Example Issue Solved
1NF	Eliminate repeating groups	Each column must have atomic values	Multiple values in one cell
2NF	Eliminate partial dependency	All non-key attributes depend on the whole primary key	Some attributes depend on part of composite key
3NF	Eliminate transitive dependency	Non-key attributes should depend only on primary key	Attributes depend on other non-key attributes



KEY TAKEAWAY : Normalization helps in reducing **redundancy**, avoiding anomalies, and ensuring **data integrity**.



5. ADVANCED NORMAL FORMS - 4NF, BCNF & DENORMALIZATION

4NF (Fourth Normal Form)

A table is in 4NF if it is in BCNF and has no multivalued dependencies.

Example Scenario :

A student can have multiple Hobbies and multiple Languages.

UNF Table

StudentID	StudentName	Hobbies	Languages
1	Ankit	Reading, Cricket	Java, Python
2	Neha	Singing, Dancing	C++, Java
3	Ravi	Gaming, Music	Python, JS

4NF Decomposition

STUDENT_HOBBIES		STUDENT_LANGUAGES	
StudentID	Hobby	StudentID	Language
1	Reading	1	Java
1	Cricket	1	Python
2	Singing	2	C++
2	Dancing	2	Java
3	Gaming	3	Python
3	Music	3	JS

Rule :

Remove multivalued dependencies.
Make every independent fact separate.

Benefit :

- No data duplication
- Better data consistency

BCNF (Boyce-Codd Normal Form)

A table is in BCNF if for every functional dependency $X \rightarrow Y$, X is a super key.

Example :

UNF / 3NF Table

EmpID	EmpName	DeptID	DeptName	DeptLocation
1	Ankit	D1	IT	Pune
2	Neha	D2	HR	Mumbai
3	Ravi	D1	IT	Pune

BCNF Decomposition

EMPLOYEE			DEPARTMENT		
EmpID	EmpName	DeptID	DeptID	DeptName	DeptLocation
1	Ankit	D1	D1	IT	Pune
2	Neha	D2	D2	HR	Mumbai
3	Ravi	D1			

Rule :

For every functional dependency $X \rightarrow Y$, X must be a super key (i.e., X uniquely identifies Y).

Benefit :

- Eliminates redundancy caused by non-key determinants.
- Stronger version than 3NF.

DENORMALIZATION

Intentionally adding redundancy to improve read performance.
Useful in reporting, analytics and very high read systems.

Example :

Normalized Tables

EMPLOYEE			DEPARTMENT	
EmpID	EmpName	DeptID	DeptID	DeptName
1	Ankit	D1	D1	IT
2	Neha	D1	D2	HR
3	Ravi	D2		

Denormalized Table
(for Reporting / Faster Read)

EmpID	EmpName	DeptID	DeptName
1	Ankit	D1	IT
2	Neha	D1	IT
3	Ravi	D2	HR

When to Denormalize ?

- Read-heavy systems
- Reporting / Dashboards
- Data warehouses
- Caching layers

Trade-off :

Better Read
Performance but
More storage and
Complex updates.

NORMALIZATION SUMMARY

Normal Form	Focus	Key Rule / Condition	Eliminates	Example Dependency Eliminated
1NF	Atomic Values	No repeating groups, atomic (single) values	Repeating groups	{A, B1, B2} $\rightarrow$ repeating B's
2NF	Partial Dependency	No partial dependency. Depends on whole key	Partial dependency	$A \rightarrow B$ (when key is {A, C})
3NF	Transitive Dependency	No transitive dependency. Non-key depends only on key	Transitive dependency	$A \rightarrow B, B \rightarrow C \Rightarrow A \rightarrow C$
BCNF	Stronger 3NF	For every $X \rightarrow Y$, X must be a super key	Non-key determinants	$B \rightarrow C$ (where B not a key)
4NF	Multivalued Dependency	No multivalued dependency except by super key	Multivalued dependency	$A \leftrightarrow B$ (B has multiple values)



KEY TAKEAWAY : Normalize to store data efficiently and accurately.
Denormalize only when you need better read performance.



6. ACID PROPERTIES & CONSTRAINTS IN DBMS

ACID Properties

ACID properties ensure that transactions are processed reliably even in case of failures.

A - Atomicity



All operations in a transaction are completed or none are.

Example :

Money transfer either complete or not at all.

C - Consistency



A transaction takes the database from one valid state to another.

Example :

Account balance remains consistent after transaction.

I - Isolation



Concurrent transactions do not interfere with each other.

Example :

Two users updating data simultaneously don't affect each other.

D - Durability



Once a transaction is committed, it remains permanent even after failure.

Example :

Committed data is saved in disk permanently.

Constraints in DBMS - Rules applied on tables to ensure data accuracy and reliability.

① NOT NULL Constraint

- Ensures a column cannot have NULL values.
- Every row must have a value in this column.

Example :

Student

ID	Name	Email (NOT NULL)
1	Ankit	ankit@gmail.com
2	Neha	neha@gmail.com
3	Ravi	NULL X

In Email column, NULL value is not allowed. So this row will be rejected.

② UNIQUE Constraint

- Ensures all values in a column (or set of columns) are different.
- Duplicate values are not allowed.

Example :

Student

ID	Email (UNIQUE)
1	ankit@gmail.com
2	neha@gmail.com
3	neha@gmail.com X

Duplicate email is not allowed as it violates UNIQUE constraint.

③ PRIMARY KEY Constraint

- Uniquely identifies each record in a table.
- Cannot be NULL and must be UNIQUE.
- Only one primary key per table.

Example :

Student

ID (PK)	Name	Email
1	Ankit	ankit@gmail.com
2	Neha	neha@gmail.com
3	Ravi	ravi@gmail.com

ID uniquely identifies each student.

④ FOREIGN KEY Constraint

- Links a column (or columns) in one table to the primary key in another table.
- Maintains referential integrity.

Example :

EMPLOYEE

DEPARTMENT

DeptID (PK)	DeptName
D1	IT
D2	HR
D3	Finance

EmpID (PK)	EmpName	DeptID (FK)
101	Ankit	D1
102	Neha	D2
103	Ravi	D1

DeptID in EMPLOYEE references DeptID in DEPARTMENT.

⑤ CHECK Constraint

- Ensures that all values in a column satisfy a specific condition.
- Helps in enforcing business rules.

Example :

Employee

EmpID	Name	Age (CHECK Age >= 18)
101	Ankit	22
102	Neha	17 X
103	Ravi	25

Age must be greater than or equal to 18. So 17 is rejected.

⑥ DEFAULT Constraint

- Provides a default value to a column if no value is specified during insertion.

Example :

Employee

EmpID	Name	Status (DEFAULT 'Active')
101	Ankit	Active
102	Neha	Inactive
103	Ravi	Active (auto)

Status not provided, so default value 'Active' is used.



KEY TAKEAWAY

- ACID properties guarantee reliable transaction processing.
- Constraints enforce rules to maintain accurate and consistent data.
- Together they ensure data integrity, reliability and consistency.



7. INDEXING IN DBMS

What is Index?

An index is a data structure that improves the **speed** of data **retrieval** operations on a database table.
(Like the index of a book)



Why Indexing?

- ✓ Speeds up SELECT queries
- ✓ Reduces disk I/O
- ✓ Improves performance of searches, joins & sorting
- ✓ Helps in enforcing uniqueness (Unique Index)

Trade-off

- Slower INSERT, UPDATE, DELETE (extra work to maintain index)
- Extra storage space
- Too many indexes can degrade performance

★ TYPES OF INDEXES

1) PRIMARY INDEX

- Built on the primary key of an ordered (sorted) file.
- Only one primary index per table.

Example :

RollNo (PK)	Name	Marks
1	Ankit	85
2	Neha	90
3	Ravi	78

Index

RollNo	Pointer
1	→
2	→
3	→

2) UNIQUE INDEX

- Built on columns that have UNIQUE constraint.
- Values must be unique but can be NULL (in most DBMS).

Example : (Email column)

Email (Unique)	Pointer
ankit@gmail.com	→
neha@gmail.com	→
ravi@gmail.com	→

3) NON-UNIQUE INDEX

- Built on columns that can have duplicate values.
- Helps in searching but duplicates are allowed.

Example : (Dept column)

Dept	Pointer List
IT	→, →
HR	→
Finance	→, →, →

4) CLUSTERED INDEX

- The actual data rows are stored in the order of the index key.
- Only one clustered index per table.
- Faster range queries.

Example : (Clustering on RollNo)

Index

RollNo	Pointer
1	→
2	→
3	→

Data (Stored in order)

RollNo	Name	Marks
1	Ankit	85
2	Neha	90
3	Ravi	78

5) NON-CLUSTERED INDEX

- Index is separate from data.
- Contains a pointer to the actual data row.
- Multiple non-clustered indexes can exist.

Example : (Index on Name)

Index

Name	Pointer
Ankit	→
Neha	→
Ravi	→

Data (Unordered)

RollNo	Name	Marks
2	Neha	90
1	Ankit	85
3	Ravi	78

★ WHEN TO CREATE INDEX?

Create Index on columns used in :

- ✓ WHERE clause
- ✓ JOIN conditions
- ✓ ORDER BY / GROUP BY
- ✓ Columns with high selectivity (many unique values)

Avoid / Don't create index on :

- ✗ Very small tables
- ✗ Columns with very low selectivity (many duplicate values)
- ✗ Columns that are frequently updated

SELECTIVITY

Selectivity = $\frac{\text{Number of unique values in a column}}{\text{Total number of rows in the table}}$

High selectivity → good for index

Low selectivity → not good for index

★ INDEXING SUMMARY

Index Type	Based On	Allows Duplicates?	NULL Allowed?	Clustered?	Use Case
Primary Index	Primary Key (Ordered file)	No	No	Yes	Default index on PK
Unique Index	Unique Column(s)	No	Yes (most DBMS)	No	Enforce uniqueness
Non-Unique Index	Non-Unique Column(s)	Yes	Yes	No	Search on non-unique data
Clustered Index	Any Column(s)	No (keys unique)	No	Yes	Range queries, sorted data
Non-Clustered Index	Any Column(s)	Yes	Yes	No	Frequent searches, joins



KEY TAKEAWAY : Indexes make data retrieval **faster** but come with a cost in storage and write performance. Use indexes **wisely** for best performance!



8. CLUSTERED vs NON-CLUSTERED INDEX - IN DETAIL

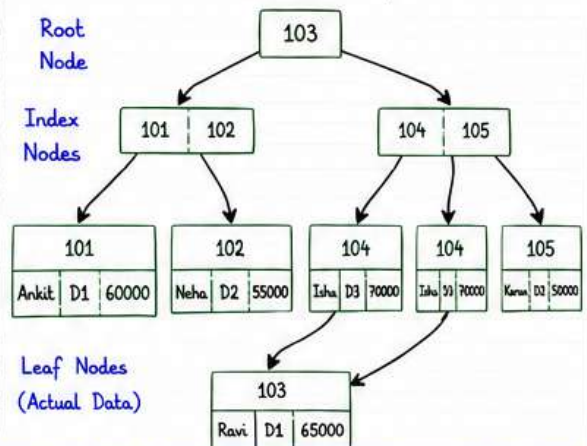
1) CLUSTERED INDEX

- The actual data rows are stored in the order of the clustered index key.
- Only one clustered index per table.
- Leaf nodes contain the actual data (not pointers).
- Great for range queries.

Example Table : Employee

EmpID (PK)	Name	DeptID	Salary
101	Ankit	D1	60000
102	Neha	D2	55000
103	Ravi	D1	65000
104	Isha	D3	70000
105	Karan	D2	50000

Clustered Index on EmpID (PK)



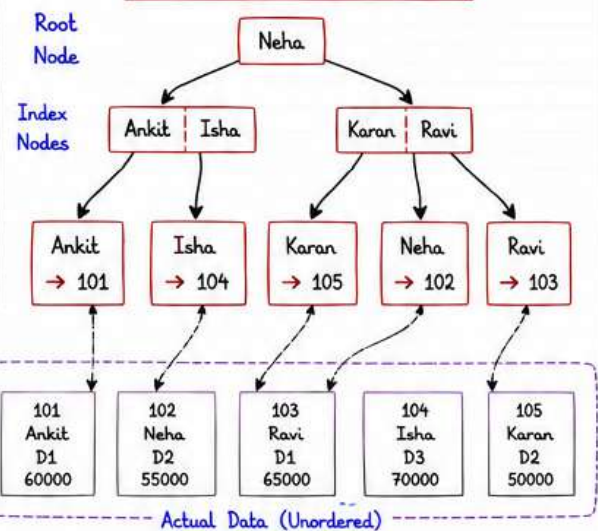
2) NON-CLUSTERED INDEX

- Index is separate from the actual data.
- Leaf nodes contain a pointer to the actual data row.
- Multiple non-clustered indexes can exist on a table.
- Good for equality searches.

Example Table : Employee

EmpID (PK)	Name	DeptID	Salary
101	Ankit	D1	60000
102	Neha	D2	55000
103	Ravi	D1	65000
104	Isha	D3	70000
105	Karan	D2	50000

Non-Clustered Index on Name



3) CLUSTERED vs NON-CLUSTERED - COMPARISON

Feature	Clustered Index	Non-Clustered Index
Data Storage	Data stored in index order	Data stored separately
Leaf Node	Contains actual data rows	Contains pointer to data rows
Number per Table	Only one	Multiple allowed
Primary Key	Usually created on PK	Can be created on any column
Range Query	Very Fast	Slower (extra lookup)
Insert / Update	Slower (maintain order)	Faster (index separate)
Storage Space	Less	More
Best Use Case	Range queries, sorting	Equality searches, filters

4) SELECTIVITY (IMPORTANT!)

$$\text{Selectivity} = \frac{\text{Number of unique values in a column}}{\text{Total number of rows in the table}}$$

- High Selectivity (closer to 1) → Good for Index
(Example: Email ID, RollNo)
- Low Selectivity (closer to 0) → Not good for Index
(Example: Gender, Status (Active/Inactive))

5) WHEN TO CREATE INDEX?

- ✓ Columns used in WHERE clause
- ✓ Columns used in JOIN conditions
- ✓ Columns used in ORDER BY / GROUP BY
- ✓ Columns with High Selectivity
- ✓ Frequently searched columns
- ✓ Foreign Key columns
- ✓ Columns used in DISTINCT

6) WHEN NOT TO CREATE INDEX?

- ✗ Very small tables
- ✗ Columns with Low Selectivity
- ✗ Columns that are frequently updated (INSERT / UPDATE / DELETE)
- ✗ Columns where index won't be used by queries
- ✗ Too many indexes on a table



Quick Tips

- Indexes improve read performance but may slow down write operations.
- Always check the query execution plan before creating an index.
- Don't over-index!
- Use the right index for the right query.



KEY TAKEAWAY

Clustered index = Data is ordered by the index.

Non-clustered index = Separate structure with pointers to data.

Use indexes wisely to balance read performance and write cost.



9. B+ TREE - CONCEPT, OPERATIONS & EXAMPLE

1) WHAT IS B+ TREE ?

- A B+ Tree is a balanced multi-way search tree.
- All values are stored in leaf nodes.
- Internal nodes store only keys to guide the search.
- Leaf nodes are linked in sorted order (range queries easier).

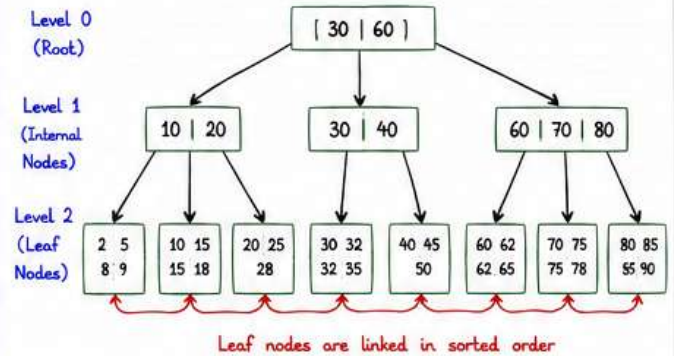
Used in DBMS Indexes.

PROPERTIES

- All leaf nodes are at the same level.
- Internal nodes have between $\lceil m/2 \rceil$ and m children.
- Leaf nodes have between $\lceil (m-1)/2 \rceil$ and $(m-1)$ keys.
- For m -way B+ tree, maximum children = m .

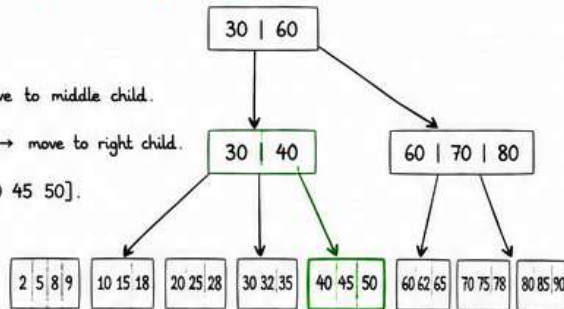
m = order of the B+ tree
(max children of any internal node)

BASIC STRUCTURE (Order = 4)



2) SEARCH in B+ TREE (Search key = 45)

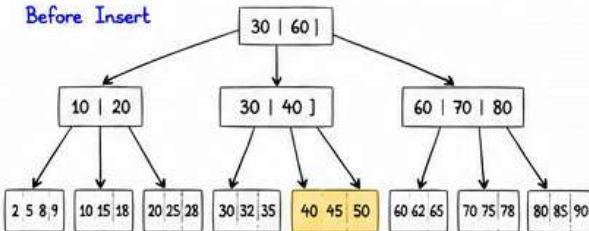
- Start from Root. Compare 45 with keys.
- $30 < 45 < 60 \rightarrow$ move to middle child.
- In $[30 | 40]$, $45 > 40 \rightarrow$ move to right child.
- Reached leaf node $[40 45 50]$. Key 45 found.



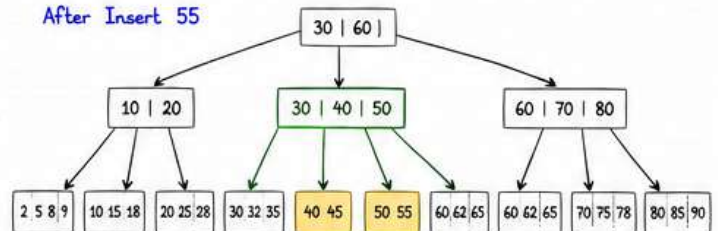
3) INSERT in B+ TREE (Insert key = 55)

- Start from Root and reach leaf node $[40 45 50]$.
- Insert 55 $\rightarrow [40 45 50 55]$ (No overflow, since max keys in leaf = $m-1 = 3$ for order 4 $\rightarrow$ after insert = 4, overflow).
- Split the leaf node.
- Promote first key of new leaf (50) to parent.
- Parent $[30 | 40]$ becomes $[30 | 40 | 50]$. (No overflow in parent)

Before Insert



After Insert 55

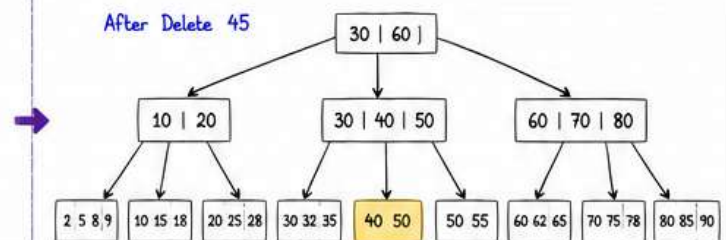


4) DELETE in B+ TREE (Delete key = 45)

- Find leaf node containing 45 $\rightarrow [40 45 50]$.
- Delete 45 $\rightarrow [40 50]$ (No underflow, min keys = $\lceil (m-1)/2 \rceil = 2$)
- If after delete, keys $<$ min keys $\rightarrow$
 - Borrow from sibling (if possible)
 - Else merge with sibling and update parent.

Before Delete

After Delete 45



5) B+ TREE vs B TREE

Feature	B+ Tree	B Tree
Where data stored?	Only in Leaf Nodes	In Internal + Leaf Nodes
Leaf node linkage	Yes (linked list)	No
Search	Always to leaf node	May end at internal node
Range Query	Very efficient (using leaf links)	Less efficient
Space utilization	Better	Comparatively less
Fan-out	Better	Lower
Used in DBMS	Yes (Indexes)	Rarely used

6) ADVANTAGES

- ✓ Balanced tree $\rightarrow O(\log n)$ search, insert, delete
- ✓ Leaf links $\rightarrow$ Range queries very fast
- ✓ Higher fan-out $\rightarrow$ Less height
- ✓ Efficient for large datasets (used in indexes)

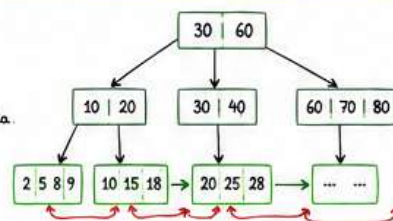
USE CASES

- Indexing in DBMS (Primary/Secondary Index)
- File systems (e.g., NTFS)
- Database storage engines (InnoDB, PostgreSQL, etc.)

SUMMARY

- 🔍 Search $\rightarrow O(\log n)$
- ⊕ Insert $\rightarrow O(\log n)$
- ⊖ Delete $\rightarrow O(\log n)$

- * All leaves at same level.
- * Only leaves store actual data.
- * Leaf nodes linked for fast range queries.



KEY TAKEAWAY

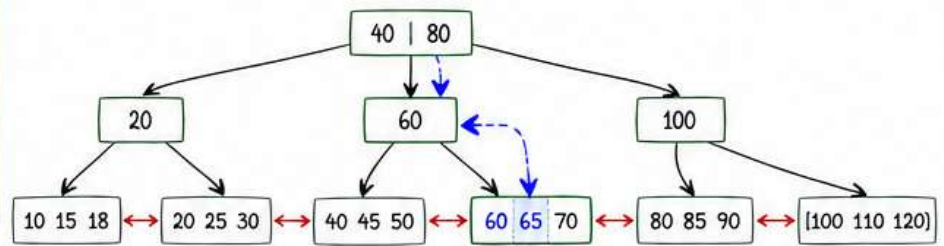
B+ Tree is the backbone of Database Indexing. If you understand its structure & operations well, many DBMS performance questions become easy in interviews!

9. B+ TREE - OPERATIONS (DRY RUN) WITH ORDER = 4

RECAP (Order = 4)

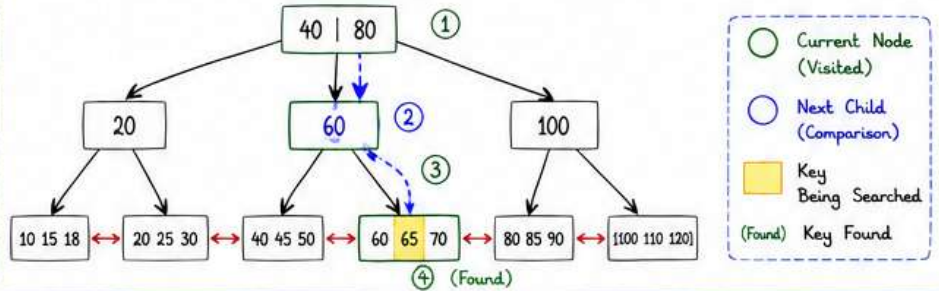
- Max keys in a node = 4
- Max children = 5
- Min keys in non-root node = $\text{ceil}(4/2) = 2$
- All leaf nodes at same level
- All actual data is stored in leaf nodes.

INITIAL B+ TREE



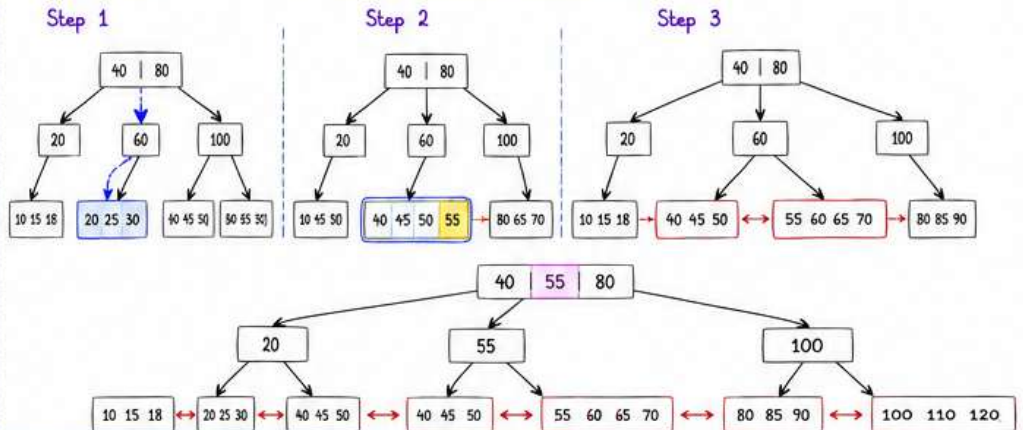
1) SEARCH (Key = 65)

- ① Start at Root.
- ② $65 > 40$ but $< 80 \rightarrow$ go to middle child [60]
- ③ $65 > 60 \rightarrow$ go to right child leaf [60 65 70]
- ④ Key 65 found in leaf node.



2) INSERT (Key = 55)

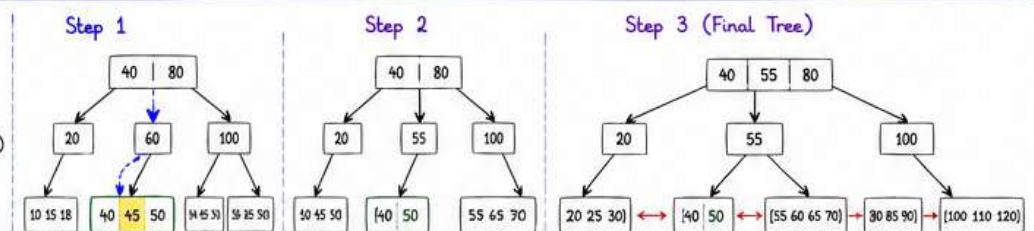
- ① Go to leaf where 55 should be.
- ② Insert 55 in leaf $\rightarrow [40 45 50 55]$
- ③ Leaf overflow (5 keys) $\rightarrow$ split leaf.
- ④ Promote first key of right leaf (55) to parent.
- ⑤ Parent becomes $\rightarrow [40 55 80]$ (OK)
- ⑥ Tree after insertion.



Note: In B+ Tree, when leaf splits, we promote the first key of right leaf to parent.

3) DELETE (Key = 45)

- ① Go to leaf containing 45.
- ② Delete 45 $\rightarrow$ leaf becomes [40 50]
- ③ Min keys required = 2 $\rightarrow$ OK (no underflow)
- ④ No redistribution / merge needed.
- ⑤ Tree after deletion.



Underflow Case Example (If leaf becomes [40])

- Try Borrow from Left $\rightarrow$ Else Borrow from Right $\rightarrow$ Else Merge with Sibling

(Not needed in above delete since min keys satisfied)

4) B+ TREE vs B TREE

Feature	B+ TREE	B TREE
Where data is stored?	Only in Leaf Nodes	In Internal + Leaf Nodes
Search ends at?	Always at Leaf Node	May end at Internal Node
Leaf Node Linkage	Yes (Linked List)	No
Range Query	Very Efficient	Less Efficient
Fan-out	Higher (more children)	Lower
Space Utilization	Better	Comparatively less
Used in DBMS?	Yes (Most DBMS)	Rarely used



5) KEY TAKEAWAYS

- B+ Tree keeps tree balanced $\rightarrow O(\log n)$ for search, insert and delete.
- All leaves at same level $\rightarrow$ predictable performance.
- Leaf linkage makes range queries super fast.
- Ideal for DBMS indexing (Primary & Secondary Index).

TIME COMPLEXITY

- 🔍 Search $\rightarrow O(\log n)$
- ⊕ Insert $\rightarrow O(\log n)$
- ⊖ Delete $\rightarrow O(\log n)$

REAL WORLD EXAMPLE



In a Bank's customer table with millions of records, B+ Tree indexes help find a customer very fast using $\log n$ operations.

11. SQL QUERY EXECUTION ORDER (VISUAL DRY RUN)

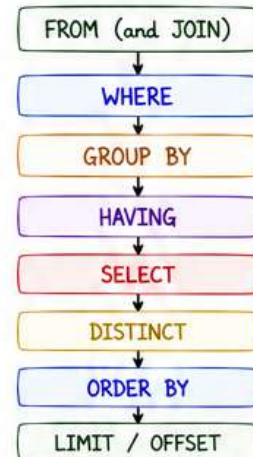
EXAMPLE QUERY

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 2
ORDER BY COUNT(*) DESC;
```

ACTUAL EXECUTION ORDER (Not the Writing Order!)

- ① FROM
- ② JOIN (if any)
- ③ WHERE
- ④ GROUP BY
- ⑤ HAVING
- ⑥ SELECT
- ⑦ DISTINCT
- ⑧ ORDER BY
- ⑨ LIMIT / OFFSET

EXECUTION FLOW



SQL Server Processes the query in this order to produce the final result.



VISUAL DRY RUN (Step by Step)

- ① FROM employees
(Load data from table)

id	name	department	salary
1	Alice	IT	60000
2	Bob	IT	45000
3	Charlie	HR	55000
4	David	IT	70000
5	Eve	HR	48000
6	Frank	Finance	62000
7	Grace	Finance	30000
8	Henry	IT	52000
9	Ivy	HR	58000
10	Jack	Finance	67000

- ② WHERE salary > 50000
(Filter rows)

id	name	department	salary
1	Alice	IT	60000
3	Charlie	HR	55000
4	David	IT	70000
6	Frank	Finance	62000
8	Henry	IT	52000
9	Ivy	HR	58000
10	Jack	Finance	67000

- ③ GROUP BY department
(Create groups)

department	Rows in Group	COUNT(*)
IT	(1, 4, 8)	3
HR	(3, 9)	2
Finance	(6, 10)	2

- ④ HAVING COUNT(*) > 2
(Filter groups)

department	COUNT(*)
IT	3

HR and Finance removed because their count is not > 2.

- ⑤ SELECT department, COUNT(*)
(Project required columns)

department	COUNT(*)
IT	3

- ⑥ (DISTINCT)
No duplicate rows, so result remains same.

department	COUNT(*)
IT	3

- ⑦ ORDER BY COUNT(*) DESC
(Sort result)

department	COUNT(*)
IT	3

(Only one row, so no change)

- ⑧ LIMIT / OFFSET (if any)
(Limit result set)

department	COUNT(*)
IT	3

FINAL RESULT

department	COUNT(*)
IT	3

This is the output returned to the user.



IMPORTANT POINTS

- Execution order is NOT the same as how we write the query.
- WHERE works on individual rows.
- HAVING works on groups (after aggregation).
- SELECT happens after filtering groups.

WHERE vs HAVING

WHERE

- Used before GROUP BY
- Works on rows
- Cannot use aggregate functions
- Improves performance

HAVING

- Used after GROUP BY
- Works on groups
- Can use aggregate functions
- Slower (after grouping)



INTERVIEW TRAPS

- Why WHERE can't use COUNT() / SUM() ?
→ Because aggregation hasn't happened yet.
- Why HAVING can use aggregates ?
→ Because groups are already formed.
- ORDER BY works on the final result set.
- LIMIT is applied at the very end.



KEY TAKEAWAY :

Understand the execution order, not just the syntax.
It helps you write correct queries and also optimize them!



12. SQL QUERY EXECUTION ORDER ON JOIN (VISUAL DRY RUN)

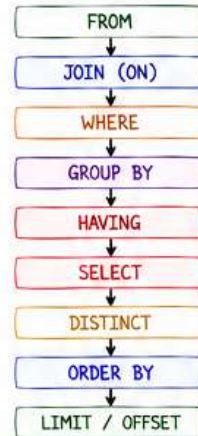
EXAMPLE QUERY

```
SELECT d.department_name, COUNT(e.emp_id) AS emp_count
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.salary > 50000
GROUP BY d.department_name
HAVING COUNT(e.emp_id) > 2
ORDER BY emp_count DESC;
```

ACTUAL EXECUTION ORDER

- 1 FROM (tables)
- 2 JOIN (ON condition)
- 3 WHERE
- 4 GROUP BY
- 5 HAVING
- 6 SELECT
- 7 DISTINCT
- 8 ORDER BY
- 9 LIMIT / OFFSET

EXECUTION FLOW



Database processes the query in this order to produce the final result.



TABLES (Sample Data)

employees (e)

emp_id	name	dept_id	salary
1	Alice	10	60000
2	Bob	10	45000
3	Charlie	20	55000
4	David	20	70000
5	Eve	30	48000
6	Frank	10	62000
7	Grace	30	52000
8	Henry	20	52000
9	Ivy	10	58000

departments (d)

dept_id	department_name
10	IT
20	HR
30	Finance

- ① FROM employees e, departments d
(Load rows from both tables)

employees (e)				departments (d)	
emp_id	name	dept_id	salary	dept_id	department_name
1	Alice	10	60000	10	IT
2	Bob	10	45000	20	HR
3	Charlie	20	55000	30	Finance
4	David	20	70000		
5	Ivy	30	48000		

(9 rows) (3 rows)

- ② JOIN e.dept_id = d.dept_id
(Inner Join)

emp_id	name	dept_id	salary	department_name
1	Alice	10	60000	IT
2	Bob	10	45000	IT
3	Charlie	20	55000	HR
4	David	20	70000	HR
5	Eve	30	48000	Finance
6	Frank	10	62000	IT
7	Grace	30	52000	Finance
8	Henry	20	52000	HR
9	Ivy	10	58000	IT

(9 rows)

- ③ WHERE e.salary > 50000
(Filter rows)

emp_id	name	dept_id	salary	department_name
1	Alice	10	60000	IT
3	Charlie	20	55000	HR
4	David	20	70000	HR
6	Frank	10	62000	IT
7	Grace	30	52000	Finance
8	Henry	20	52000	HR
9	Ivy	10	58000	IT

(7 rows)

- ④ GROUP BY d.department_name
(Create groups)

department_name	Rows in Group	emp_ids
IT	{1,6,9}	3
HR	{3,4,8}	3
Finance	{7}	1

- ⑤ HAVING COUNT(emp_id) > 2
(Filter groups)

department_name	COUNT(emp_id)
IT	3
HR	3

(Finance removed because count = 1)

- ⑥ SELECT d.department_name, COUNT(e.emp_id) AS emp_count
(Project required columns)

department_name	emp_count
IT	3
HR	3

- ⑦ DISTINCT (No duplicates here, so result remains same)

department_name	emp_count
IT	3
HR	3

- ⑧ ORDER BY emp_count DESC
(Sort result)

department_name	emp_count
IT	3
HR	3

(Both have same count, so order may remain as is)

- ⑨ LIMIT / OFFSET (Not used here)

department_name	emp_count
IT	3
HR	3

FINAL RESULT

department_name	emp_count
IT	3
HR	3

This is the output returned to the user.

WHERE vs HAVING

WHERE

- Works on individual rows before grouping.
- Cannot use aggregate functions.
- Improves performance (filters early).

HAVING

- Works on groups after grouping.
- Can use aggregate functions.
- Slower (filters later).

INTERVIEW TRAPS

- Why WHERE can't use COUNT()?
→ Because aggregation hasn't happened yet.
- Why HAVING can use COUNT()?
→ Because groups are already formed.
- SELECT happens after HAVING (not before).
- ORDER BY is applied on the final result set.
- LIMIT is the very last step.



KEY TAKEAWAY

- ✓ Understand the execution order, not the writing order.
- ✓ Joins happen before filtering (WHERE).
- ✓ Grouping happens before SELECT.
- ✓ Correct order = Correct result + Better performance!

13. SQL JOINS - ALL TYPES (VISUAL GUIDE)

EMPLOYEES (E)

EmpID	EmpName	DeptID
101	Alice	10
102	Bob	20
103	Charlie	30
104	David	NULL

DEPARTMENTS (D)

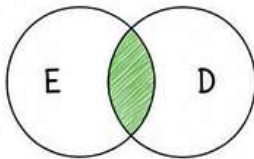
DeptID	DeptName
10	IT
20	HR
30	Finance
40	Marketing

Legend

- = Matching rows
- = From Left (E) only
- = From Right (D) only
- = No Match (NULL)

1) INNER JOIN

Returns only matching rows from both tables.

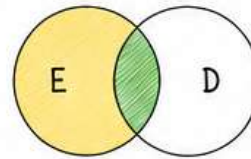


```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance

2) LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from LEFT table (E) and matching rows from RIGHT (D). Non-matching from D → NULL.

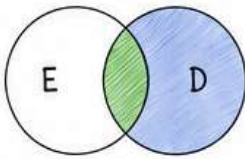


```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
LEFT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
104	David	NULL

3) RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from RIGHT table (D) and matching rows from LEFT (E). Non-matching from E → NULL.

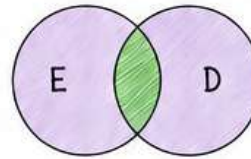


```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
RIGHT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
NULL	NULL	Marketing

4) FULL OUTER JOIN

Returns all rows from both tables. Match → values shown. No match → NULL on missing side.



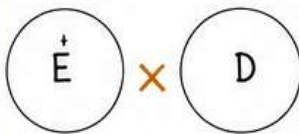
```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
FULL OUTER JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
104	David	NULL
NULL	NULL	Marketing

5) CROSS JOIN

Returns Cartesian Product (every row of E with every row of D).

Rows = |E| × |D| = 4 × 4 = 16



Every combination

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
CROSS JOIN Departments D;
```

EmpID	EmpName	DeptName
101	Alice	IT
101	Alice	HR
101	Alice	Finance
101	Alice	Marketing
...	...	...
104	David	Marketing

(Total 16 rows)

6) SELF JOIN

Joins a table to itself.

Example: Employee → Manager

Dept 10 (IT)
101 Alice (Lead)
↳ 102 Bob
↳ 103 Charlie

Dept 20 (HR)
201 Emma (Lead)
↳ 202 Frank

```
SELECT E.EmpID, E.EmpName,
M.EmpName AS ManagerName
FROM Employees E
LEFT JOIN Employees M
ON E.DeptID = M.DeptID
AND E.EmpID <> M.EmpID;
```

EmpID	EmpName	ManagerName
102	Bob	Alice
103	Charlie	Alice
101	Alice	Bob, Charlie
202	Frank	Emma
201	Emma	Frank

JOIN COMPARISON SUMMARY

JOIN TYPE	RETURNS	MISSING ROWS	INTERVIEW USE
INNER JOIN	Only matching rows	No missing rows	Most common
LEFT JOIN	All Left + Matches	Right → NULL	Very common
RIGHT JOIN	All Right + Matches	Left → NULL	Less common
FULL OUTER JOIN	All rows from both	NULL both sides	Rare
CROSS JOIN	Every combination	N/A	Cartesian Product
SELF JOIN	Table joined with itself	N/A	Hierarchy, reports

INTERVIEW QUESTIONS (TOP ASKED)

- What is the difference between INNER JOIN and LEFT JOIN?
- Can LEFT JOIN return duplicate rows? When?
- Why is CROSS JOIN dangerous?
- Can INNER JOIN return NULL values?
- What is the difference between ON and WHERE?
- In what order are JOINS executed in a query?
- Which JOIN is the fastest? Why?
- How does MySQL execute JOINS internally?



KEY TAKEAWAY

- ✓ Choose the correct JOIN based on what data you want to keep.
- ✓ ON is used to match rows. WHERE is used to filter rows after join.
- ✓ Understand NULL handling in OUTER JOINS.
- ✓ Always try to write JOINS before subqueries for better performance.



QUICK TIP

Think: Do I need all rows from a table?
→ LEFT / RIGHT / FULL JOIN
Do I need only matching rows?
→ INNER JOIN
Do I need all combinations?
→ CROSS JOIN

13. SQL JOINS - ALL TYPES (VISUAL GUIDE)

EMPLOYEES (E)

EmpID	EmpName	DeptID
101	Alice	10
102	Bob	20
103	Charlie	30
104	David	NULL
105	Eve	40

DEPARTMENTS (D)

DeptID	DeptName
10	IT
20	HR
30	Finance
40	Marketing
50	Operations

LEGEND

- = Matching Rows
- = From Left (E) only
- = From Right (D) only
- = No Match (NULL)

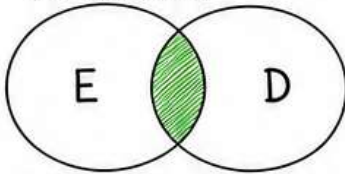


QUICK RULES

- INNER → Only matches from both tables.
- LEFT → All from LEFT + matches.
- RIGHT → All from RIGHT + matches.
- FULL → All from BOTH + non-matches.
- CROSS → Every combination.
- SELF → Table joined with itself.

1) INNER JOIN

(Only matching rows from both tables)



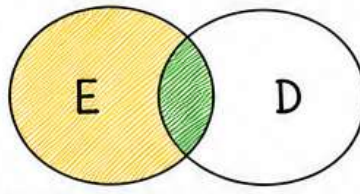
```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
105	Eve	Marketing

(No NULLs from either side)

2) LEFT JOIN (LEFT OUTER JOIN)

(All from LEFT + matches from RIGHT)



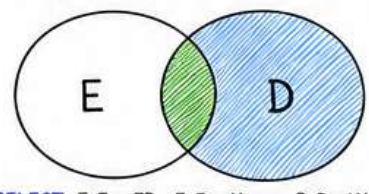
```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
LEFT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
104	David	NULL
105	Eve	Marketing

(Non-matching from E → NULL)

3) RIGHT JOIN (RIGHT OUTER JOIN)

(All from RIGHT + matches from LEFT)



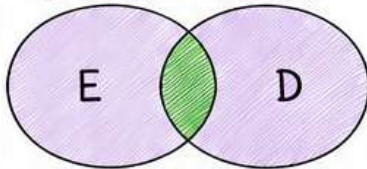
```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
RIGHT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
105	Eve	Marketing
NULL	NULL	Operations

(Non-matching from D → NULL)

4) FULL OUTER JOIN

(All rows from BOTH tables)



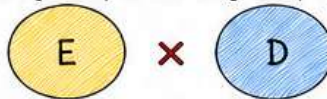
```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
FULL OUTER JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	EmpName	DeptName
101	Alice	IT
102	Bob	HR
103	Charlie	Finance
104	David	NULL
105	Eve	Marketing
NULL	NULL	Operations

(Non-matching on both sides → NULL)

5) CROSS JOIN

(Cartesian Product
Every row of E with every row of D)



$$\text{Rows} = |E| \times |D| = 5 \times 5 = 25$$

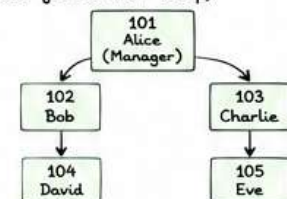
```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
CROSS JOIN Departments D;
```

EmpID	EmpName	DeptName
101	Alice	IT
101	Alice	HR
101	Alice	Finance
...	...	...
105	Eve	Operations

(All possible combinations)

6) SELF JOIN

(Table joined with itself)



```
SELECT E.EmpID, E.EmpName, M.EmpName AS ManagerName
FROM Employees E
LEFT JOIN Employees M
ON E.ManagerID = M.EmpID;
```

EmpID	EmpName	ManagerName
101	Alice	NULL
102	Bob	Alice
103	Charlie	Alice
104	David	Bob
105	Eve	Charlie

(Common for hierarchy / reporting)

JOIN COMPARISON SUMMARY

JOIN TYPE	RETURNS	MISSING ROWS	NULLS FROM	INTERVIEW USE
INNER JOIN	Only matching rows	No missing rows	None	Most Common
LEFT JOIN	All Left + Matches	Right side	Right → NULL	Very Common
RIGHT JOIN	All Right + Matches	Left side	Left → NULL	Less Common
FULL OUTER JOIN	All rows from both	None	Both sides	Rare
CROSS JOIN	Every combination	N/A	N/A	Cartesian Product
SELF JOIN	Table joined with itself	N/A	N/A	Hierarchy, Reports

INTERVIEW QUESTIONS (TOP ASKED)

1. What is the difference between INNER JOIN and LEFT JOIN?
2. Can LEFT JOIN return duplicate rows? When?
3. Why is CROSS JOIN dangerous?
4. Can INNER JOIN return NULL values?
5. What is the difference between ON and WHERE?
6. In what order are JOINS executed in a query?
7. Which JOIN is the fastest? Why?
8. How does MySQL execute JOINS internally?
9. What is SELF JOIN? Give an example use case.
10. When should we avoid using FULL OUTER JOIN?



KEY TAKEAWAY

- ✓ Choose the right JOIN based on data you want to keep.
- ✓ ON → Define match condition. WHERE → Filter after join.
- ✓ Understand NULL behavior in OUTER JOINS.
- ✓ Write JOINS first, then apply filters for better performance.



Pro Tip:

Always visualize the JOIN before writing the query!

14. UNION vs UNION ALL (VISUAL GUIDE)

WHAT ARE THEY?

UNION → Combines results of two or more SELECT queries and removes duplicates.

UNION ALL → Combines results of two or more SELECT queries and keeps duplicates.



IMPORTANT RULES

- Both queries must have same number of columns.
- Corresponding columns must be of compatible data types.
- Column names in result come from the FIRST query.

EXAMPLE TABLES

Students_A

ID	Name
1	Alice
2	Bob
3	Charlie
4	David

Students_B

ID	Name
3	Charlie
4	David
5	Eve
6	Frank

Notice: Rows with ID 3 and 4 are common in both tables.

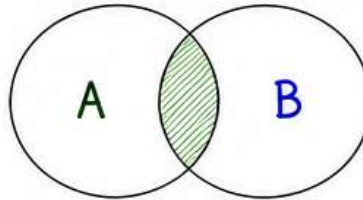
MULTIPLE QUERIES?

You can combine more than two SELECT statements.

Example (UNION):

```
SELECT ID, Name FROM A
UNION
SELECT ID, Name FROM B
UNION
SELECT ID, Name FROM C;
(All results are unique.)
```

1) UNION (REMOVES DUPLICATES)



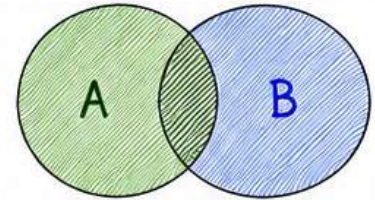
Returns: Only unique rows

```
SELECT ID, Name FROM Students_A
UNION
SELECT ID, Name FROM Students_B;
```

RESULT (UNION)

ID	Name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve
6	Frank

2) UNION ALL (KEEPS DUPLICATES)



Returns: All rows including duplicates

```
SELECT ID, Name FROM Students_A
UNION ALL
SELECT ID, Name FROM Students_B;
```

RESULT (UNION ALL)

ID	Name
1	Alice
2	Bob
3	Charlie
4	David
3	Charlie
4	David
5	Eve
6	Frank

Duplicates are also included

QUICK COMPARISON

Feature	UNION	UNION ALL
Duplicates	Removed	Kept
Performance	Slower	Faster
Sorting / Hashing	Required	Not Required
Use Case	When unique results needed	When all rows needed
Result Size	Smaller or Equal	Greater or Equal
Default Behavior	Not Default (Must write)	

PERFORMANCE COMPARISON

UNION (Slower)

- Removes duplicates.
- Performs sorting or hashing internally.
- More CPU & memory consumption.



UNION ALL (Faster)

- Does not remove duplicates.
- Simply appends the results.
- Less CPU & memory consumption.



WHEN TO USE?

Use UNION when:

- You want only distinct (unique) rows.
- Duplicates are not allowed in the final result.

Use UNION ALL when:

- You want all rows including duplicates.
- Performance is important.
- Data is already known to be unique.

INTERVIEW QUESTIONS (TOP ASKED)

- What is the difference between UNION and UNION ALL?
- Does UNION keep duplicates?
- Which one is faster: UNION or UNION ALL? Why?
- Are column names in the result taken from first or second query?
- Can we use ORDER BY with UNION / UNION ALL?
- What happens if both queries return same rows?
- Can we use different number of columns in UNION?
- How does UNION remove duplicates internally?
- Can we use DISTINCT with UNION ALL?
- Give a real world use case of UNION and UNION ALL.



LAST MINUTE DBMS CHEAT SHEET

- ✓ Know all JOIN types (INNER, LEFT, RIGHT, FULL, CROSS, SELF)
- ✓ Execution Order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
- ✓ WHERE filters rows, HAVING filters groups
- ✓ NULL ≠ 0, NULL is UNKNOWN
- ✓ Indexes improve read performance
- ✓ Normalization reduces redundancy
- ✓ Transactions ensure ACID properties
- ✓ PRIMARY KEY → Unique + Not Null
- ✓ FOREIGN KEY → Maintains referential integrity



KEY TAKEAWAY

- Use UNION for unique results.
- Use UNION ALL for all results (including duplicates).
- UNION is safe but slower.
- UNION ALL is fast and simple.
- Pick based on requirement, not habit!



Remember: Write correct SQL → Get correct data → Impress in interview!
Practice more, Write more, Debug more!